

Learning and Practicing Logic Circuits: Development of a Mobile-based Learning Prototype

Mazyar Seraj*

Institute of Computer Science, University of Bremen, 28359 Bremen, Germany
seraj@uni-bremen.de

ABSTRACT

Nowadays, with the advent of electronic devices in everyday life, mobile devices can be utilized for learning purposes. When designing a mobile-based learning application, a large number of aspects should be taken into account. For the present paper, the following aspects are of special importance: first, it should be considered how to represent information; second, possible interactions between learner and system should be defined; third – and depending on the second aspect – it should be considered how real-time responses can be provided by the system. Moreover, psychological theories as for example the 4C/ID model and findings with respect to blended learning environments should be taken into account. In this paper, a mobile-based learning prototype concerning the learning topic "logic circuit design" is presented which considers the mentioned aspects to support independent practice. The prototype includes four different representations: (i) code-based (Verilog hardware description language), (ii) graphical-based (gate-level view), (iii) Boolean function, and (iv) truth table for each gate. The proposed learning system divides the learning content into different sections to support independent practice in meaningful steps. Multiple representations are included in order to foster understanding and transfer. The resulting implications for future work are discussed.

CCS CONCEPTS

• **Human-centered computing** → *Information Visualization*; • **Applied computing** → **Computer-assisted instruction**; *Interactive learning environments*.

KEYWORDS

mobile-based learning, blended learning, 4C/ID instructional model, multiple representations, logic circuit design

ACM Reference Format:

Mazyar Seraj. 2021. Learning and Practicing Logic Circuits: Development of a Mobile-based Learning Prototype. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts (CHI '21 Extended Abstracts)*, May 8–13, 2021, Yokohama, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3411763.3451720>

* Author's address: Mazyar Seraj is currently affiliated with the Department of Mathematics and Computer Science (M&CS), Eindhoven University of Technology (TU/e), 5612 AZ Eindhoven, Netherlands: m.seraj@tue.nl.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '21 Extended Abstracts, May 8–13, 2021, Yokohama, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8095-9/21/05...\$15.00

<https://doi.org/10.1145/3411763.3451720>

1 INTRODUCTION

Logic circuits are one of the basic subject areas in computer science. Students study different types of gates and their functions, and learn how to make larger circuits from these gates [10]. In this subject area, students are confronted with the requirement to understand and integrate different representations of logic circuits [10, 20]. Code-based and graphical models of circuits, as well as Boolean functions and truth tables are used to foster understanding of the concept of concurrency in hardware programming languages [20]. Despite several similarities, there is a big fundamental difference between software programming languages and hardware description languages. Most programming languages are inherently procedural, i.e. they run processes sequentially. For example, assume that A and B are two consecutive processes. Process B is run only if process A has been finished. On the other hand, hardware description languages work concurrently, i.e. they can execute multiple parallel processes (such as flip-flops and adders) independent of each other. For example, process A and B can be executed in parallel and independently if a change happens in their inputs.

While circuits can be graphically visualized, for example, by interconnected transistors or gates, it is also possible to describe them through Boolean functions, truth tables, or hardware-based programming languages. For learners, recognizing and understanding the complexity of hardware programming is a major challenge, especially with respect to the concept of concurrency [4, 22]. As understanding can also be supported by practical experience, electronic-based learning (e-learning) seems to be a promising approach [5]. E-learning technology offers different options such as web-based and mobile-based learning (m-learning) environments; especially m-learning becomes a promising area of research to circumvent limitations of learning time and learning place [21].

Mobile devices have become an indispensable instrument in student life. These devices can be used for learning and practicing. Mobile devices can be used to attract users and stimulate their motivation via interacting with the learning environment [23, 24]. In an educational sphere, mobile devices can encourage learners to practice due to constant access and no demand for other equipment or any special technical requirements [23, 25]. However, the challenge is how to deliver learning content to the users in order to support motivation and learning outcome optimally. With this regard, it is important to focus on ways to visualize information and to design interactions between learner and system [12, 26].

In this study, we discuss the development method of an m-learning prototype to visualize information within logic circuit design. The prototype was designed to deliver information according to the Four-Component Instructional Design model 4C/ID [30]. Four different aspects of logic circuit design are presented: (i) code-based (Verilog hardware description language [28]), (ii) graphical-based

(gate level view), (iii) Boolean function, and (iv) truth table for each gate. In addition, supportive information and interaction are considered, as they can support learners' understanding and performance [1, 16, 29]. The proposed approach covers the following criteria for an m-learning prototype:

- Based on the 4C/ID model, information is delivered within three different learning sections, providing four different representations.
- A high level of interaction is accomplished through using interactive multimedia elements [14].
- Learning hints appear in different sections and they are presented to the learners in real time [2, 14].
- Independent practice is increased in order to reduce the dependency on instructors during practice [6].

The contribution of this paper is broken down into two major parts:

1. An independent practicing style is supported by the proposed learning prototype which can be used to practice any-time and anywhere, without requiring any special technical knowledge or equipment.
2. The Learning prototype is separated in three different sections "*tutorial*", "*practice*", and "*self-evaluation*" which are based on the 4C/ID instructional model. The "*tutorial*" section is designed to provide supportive information. The "*practice*" and the "*self-evaluation*" sections are providing different levels of learning tasks. Furthermore, real time information is provided in all sections in order to support learners.

The paper is structured as follows: The section "*Background and Related Work*" provides a brief overview on Verilog and blended learning, and it is discussed how the 4C/ID instructional model can be used in an instructional m-learning application. Furthermore, the importance of multiple representations and interaction, as well as existing m-learning applications are described and discussed in this section. The "*Development Method*" section describes the method of information visualization in our approach and its application in an educational setting. The designed and developed learning prototype is presented in the section "*Learning Application*". The paper closes with "*Conclusion and Future Work*".

2 BACKGROUND AND RELATED WORK

First, the Hardware Description Language (HDL) Verilog is presented. Specifically, Verilog was chosen because it is a prevalent HDL [20, 28]. As the main emphasis of this paper is to reveal possibilities how to improve independent practice in blended e-learning environments using m-learning applications, the second part of the literature review is dedicated to blended learning. Subsequently, the instructional model 4C/ID, and the importance of multiple representations and interactions are described, as they are the main factors discussed in this paper to support independent practice. Finally, related m-learning applications are discussed in this section.

2.1 Verilog

Verilog [28] is an HDL which is used to describe digital systems. All descriptions of hardware – from switch to system level – can be supported by this language. In Verilog, modules describe hardware components. Within a module, a hardware component can be

described via a list of selectors, inputs and outputs, concurrent assignments, component instances and as procedural blocks. Several modules can be defined hierarchically to build other hardware structures. Verilog simulation environments provide graphical front-end programs as well as waveform editing [20, 28].

2.2 Blended Learning

E-learning tools provide a flexible and unbounded educational environment which is not dependent on time and place. However, this does not mean the demise of face-to-face (F2F) classrooms; it makes us to think about the best way of utilizing both e-based and F2F learning for the purpose of getting better performance [6, 11]. In contrast to other forms of e-learning activities such as fully online learning activities and online enhanced classrooms [6], blended learning is a contemplative mixture of F2F classrooms with e-learning experiences, combining learning via F2F classrooms with synchronous and asynchronous e-learning environments. A large part of the benefits of blended learning are caused by the potential of interactions in e-learning technologies. Furthermore, the success of blended learning can be attributed to independent learning activities in e-learning environments [6]. Blended learning can be beneficial to meet specific learning requirements; for instance, introducing a new concept to students may need to have F2F classrooms whereas an e-learning environment can be helpful to provide more examples and practice.

2.3 4C/ID Instructional Model

The Four-Component Instructional Design model 4C/ID was developed in order to support life-long learning [30]. The 4C/ID model helps learners to have an efficient learning experience [27]. The model considers four components: (i) learning tasks, (ii) supportive information, (iii) just-in-time (JIT) information, and (iv) part-task practice [30]. The learning tasks provide the core of the learning process. In this sense, the learning tasks need to be designed in different levels in terms of difficulties. Learning tasks are presented to the learners on the basis of supportive information. Part-task practice, in contrast, is not an obligatory element of the model and is provided only as specific routine aspects of the whole-task [27, 30]. Learners need supportive information which should be presented as tutorials so that they can master learning tasks; procedural information (JIT information) are presented to the users as a set of learning hints to guide them during working on a task.

2.4 Multiple Representations and Interactions

Most learning contents can be displayed in various ways. While different representations can help learners to gain a comprehensive and broad understanding of the subject, it is also quite demanding to integrate them in one consistent mental picture. Several studies discuss benefits of multiple representations for learning outcomes [1]. For example, expository text along with a graphic is more effective than text only to represent a topic; in general, using multiple ways to present a topic is advantageous for learners as it helps them to understand and memorize information in their long term memory [17, 18]. Interactions with different representations can improve learners' understanding and enhance their performance [1, 16]. Learners usually prefer interaction methods

that they are already familiar with, such as interactive graphical user interfaces; they are not inclined to consider new methods, even though they might find them useful and natural [13]. In other words, the small size of keypads in mobile devices imposes restrictions on designers to avoid learners' text inputs in m-learning applications [13, 23]. Furthermore, the size of interactive graphical objects must be designed accordingly so that they can be simply tapped by users [13]. In this sense, it is important to design m-learning applications in a way to deliver information using different representations with an emphasis on interaction.

2.5 Related M-learning Applications

The keywords "*Logic Circuits*" and "*Hardware Description Language*" led to nearly 40 applications, which are available in Google Play. All these applications are in the field of logic circuits, HDLs, and digital logic design. We had a closer look into these applications in terms of representation method of information, real-time response, the role of learners and instructors in the learning process (independent practicing style), and level of interaction. The majority of existing applications (e.g., Logic Circuit Simulator Pro [3], and Smart Logic Simulator [31]) allow the users to practice and design various logic circuits, using combinations of different components such as logic gates, inputs, outputs, and wires for connecting the components. Users are mostly enabled to save, reload, redesign, and rename their built circuits in these applications. There are also applications, which are focusing on the explanation of logic circuits (e.g., Digital Circuits [9]) or HDLs (e.g., VHDL and Verilog [8]). The users can learn about logic circuits and hardware description languages in a mostly written form with a minimum level of interaction.

The result contributes to the presenting of the proposed learning system. The system promoted an independent practicing style for students to have a better performance in the area of logic circuits and hardware description languages (in this case, Verilog). Furthermore, this approach is following the 4C/ID instructional model, which has an emphasis on promoting learners' schema construction for learning complex subjects. Presenting the information in three different sections named "*tutorial*", "*practice*", and "*self-evaluation*" based on the 4C/ID model in regard to the acquisition of integrated sets of knowledge on logic circuits.

In summary, the majority of existing m-learning applications are focusing on a graphical view of a logic circuit. In contrast, in the proposed approach, multiple representations are the strength compared to the other applications. Multiple representations are used in order to demonstrate more supportive information in different sections for students in real time. Multiple representations which are performed differently from one hardware program to another (e.g., logic gates and Multiplexers) is an example for this approach.

3 DEVELOPMENT METHOD

In this section, the development method of an m-learning prototype is presented. The prototype aims to support independent practice in the domain of logic circuit design within a blended learning environment; thus, the prototype is a supplementary tool apart from F2F classrooms. To this end, mainly the following factors are considered with respect to the limitations of screen and keypad size in mobile devices: interactive multimedia elements, real-time response, and

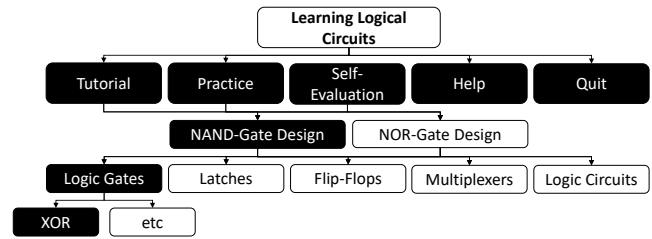


Figure 1: Learning prototype structure.

supportive information. The prototype displays four different representations of logic circuit design: (i) code-based (Verilog hardware description language); (ii) graphical-based (gate level view); (iii) Boolean function; and (iv) truth table for each gate [15]. These four aspects are visualized in three different sections according to the 4C/ID instructional model as explained in the following.

3.1 4C/ID Model and M-learning Prototype Structure

In recent years, much has been written about the potential value of implementation of 4C/ID instructional model into e-learning environments, but little research has been conducted in this area [19]. The proposed approach provides support for the notation that how this model can be applied to m-learning applications and may facilitate knowledge acquisition in the area of logic circuits. With this regard, the m-learning prototype was developed based on the design structure shown in Figure 1. Each sub-topic under the main topic (learning logic circuits) is shown as a button at the Intro page of the prototype. The learning prototype is divided into three different sections called "*tutorial*", "*practice*", and "*self-evaluation*" based on the 4C/ID model. The "*tutorial*" section presents supportive information to the users. The "*practice*" and "*self-evaluation*" sections present two learning tasks which differ in terms of difficulty and feedback. Learners can start with the "*tutorial*" and then proceed to the "*practice*" and "*self-evaluation*" parts. However, the learning system also allows the learners to choose any of these three sections in any order. Procedural information and learning hints appear in different sections and they are presented to the learners in real time. In all three sections, learners can choose between the learning content NAND gate and the learning content NOR gate. The decision for these concrete gates is based on the fact that all logic circuits can be designed with only NAND and NOR gates, and actually using only NAND and NOR gates is also beneficial with respect to manufacturing time and cost of a circuit [7]. Thus, in this prototype, all circuits are displayed using only NAND and NOR gates. Tapping on the NAND or NOR gate designs leads learners to the next page which provides different kinds of logic circuits. In this prototype, active buttons are shown with black background (see Figure 1).

3.2 Application in an Educational Setting

Our proposed approach (which is an m-learning application according to the 4C/ID model) is based on the students' independent practicing outside of F2F classrooms. The possible learning activities and the application of our approach in a blended learning environment are described in the following. Our approach is to

support independent practicing through both e-learning and F2F classes. We propose a learning system as a reference to construct the learning environment. The blended learning style environment based on our approach has four characteristics which are:

- Students can implement the practical part of instruction in the lab.
- Instructors can deliver the practical part of their lecture and additional learning content in F2F classrooms.
- Students can study the basic learning contents via the m-learning application.
- Basic part of the lecture can be presented and available for students via the m-learning application.

Design of F2F Lecture Contents. We need to use an online database to host m-learning content. The process of authoring courseware includes the development of learning materials (examples and tests) by the course instructor. The design of learning content for F2F classrooms includes the course plan and proportion of class theory and lab practical. It also includes the duration of the course, and designing tests and assignments. The purpose of the learning process is designed to strengthen the predictability of the learning process.

Design of m-learning Contents. The basis learning content is presented by the instructor and the student must first log in to the system to use the environment. After authorization, the student has to choose the subject such as Logic Gates that they tend to learn and practice (see Figure 1). Each subject consists of several examples ("tutorial" and "practice") and tests ("self-evaluation"). Each test contains problem-based questions such as easy-pick assignment.

Assessment. At the beginning and the end of each lecture, an assessment of students should be conducted. The purpose of this assessment is to find out students' performance and the potential learning outcome (what students learned from the learning application). Moreover, students' attitudes towards the hardware programming languages should be measured to find out to what extent students were accepting the m-learning application based on the blended learning strategy (which integrates independent practicing via m-learning environment).

3.3 Teaching Environment

The first step is for instructors to write Verilog code based on restrictions of the learning system. For instance, the following four constraints are given due to screen size limitations of mobile devices and to improve the quality of the graphical view of the Verilog code: (i) no more than 20 lines of code, (ii) no more than 6 inputs, (iii) no more than 10 gates, and (iv) no more than 3 inputs for each gate. Furthermore, gate, input, and output names are confined to make parsing the code easier and faster for the system. To practice independently in a blended learning environment, the learners can interact with the learning system, and experience the content provided by their instructor as well as supportive information provided by the learning system. In short, in addition to already existing Verilog files in the learning system, instructors are allowed to write Verilog code which describes a gate level circuit in a Verilog file (.v). The Verilog file can be uploaded into the system by the instructor after logging in to the system. Thereafter, students can load the file in their own application and experiment with it independently.

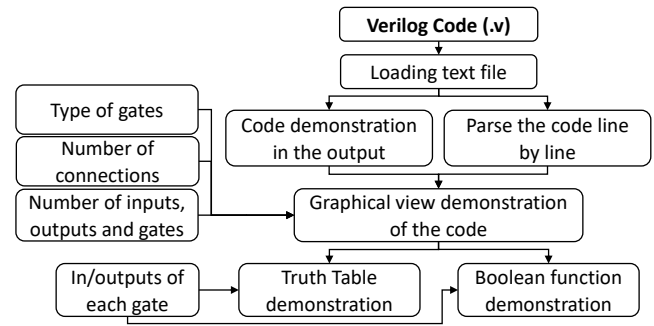


Figure 2: Learning system working diagram.

As shown in Figure 2, the prototype reads the file, parses the code, and makes a graphical view of the code including inputs, outputs, gates, and connections. Furthermore, Boolean functions and truth tables are demonstrated. Figure 2 shows a full diagram explanation about how the learning system is working. However learning system is designed to parse different input code, instructors are encouraged to follow defined rules in their coding style. For instance, the input and selector names should be defined in two different lines. The file is loaded into the learning system and it is demonstrated to the user while it is read line by line by the system (in code representation). In the "tutorial" and "practice" sections, Boolean functions and truth tables appear when the learner taps on a gate. The code and graphical representations are available in all three sections ("tutorial", "practice", and "self-evaluation"), while truth tables and Boolean functions are removed in the "self-evaluation" section to make this part more challenging for learners.

3.4 Graphical View and Connections

One main aspect of the prototype is to design an accurate graphical representation of Verilog code written by an instructor. To this end, the inputs, outputs, types and names of the gates were designed as vector images. They are stored into a library as graphical objects with a unique class name which enables to make instances from them in real time. The objects can be generated by the system every time the parser reaches one of the keywords in the code, as for example input, output, nand, or nor. After parsing the code and fetches the keywords, instances from the graphical objects will be created. A unique x-dimension and y-dimension are assigned to each defined object. Connections are created in an animated way via gradually coloring paths; thus, based on a common mobile screen size, a grid platform is designed with 4982 cells including 94 columns and 53 rows behind the created graphical objects. A unique cell is allocated to each unique point and a connection is made between the cells which need to be connected based on the Verilog code. The connection is made between each begin and end objects with coloring the cells between them.

3.5 Boolean Function and Truth Table

Complementing the set of multiple representations provided to the learners, truth tables and Boolean functions are presented to show the functionality of each gate. Users can tap on a gate and

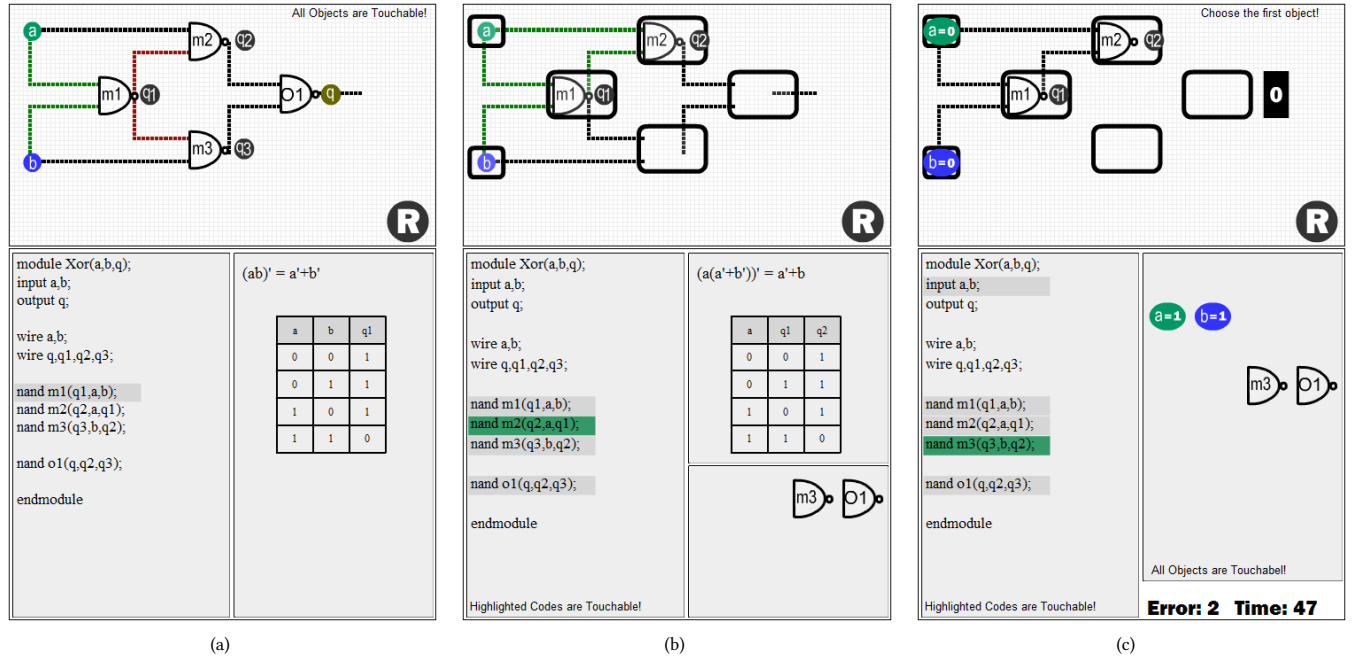


Figure 3: (a) Tutorial page, (b) Practice page, and (c) Evaluation page.

the related truth table and Boolean function will appear, according to the functionality of the gate, and input and output names.

4 LEARNING APPLICATION

In this paper, we provide a method of delivering information in an m-learning prototype in order to increase independent practice in a blended learning environment¹. The m-learning prototype is designed and developed based on the 4C/ID model to deliver information on basic logic circuits in three different sections. In sum, the prototype enables learners to learn and practice on the topic, and to evaluate their learning progress.

4.1 Different Sections of the Application

"Tutorial", "practice" and "self-evaluation" are designed to present supportive information and different levels of learning tasks in the 4C/ID model. Users can choose the button for any of these learning steps at any time of the learning process. For instance, (i) learning about logic circuits in the "tutorial" section through viewing a graphical representation of the Verilog code in an animated form via gradually coloring paths between the begin and end cells; (ii) choose the practice button to practice on logic circuits and to receive procedural information (JIT information) as guidance; (iii) evaluating their learning progress in the "self-evaluation" section. The three different sections are described in the following.

4.1.1 Tutorial. One of the main features of this prototype is to demonstrate connections between inputs and gates in an animated

form based on the Verilog code. With this regard, the concurrency of hardware programming is exemplified. The users can touch each object in the graphical representation (as shown in the hint at the top right of the graphical representation), and the system will give information to that particular object (see Figure 3(a)): (i) the corresponding line of code is highlighted, (ii) Boolean functions and truth tables are given for gates, and (iii) input connections to the particular objects are shown in green color and outputs are demonstrated in red.

4.1.2 Practice. At the beginning of the "practice" section, a popup window gives a short textual explanation. The learners are given information in four parts: (i) truth table and Boolean functions, (ii) movable gates demonstration, (iii) uncompleted graphical view of the circuit, and (iv) code view of the circuit. The correct connections are drawn between the graphical objects upon tapping on the "OK" button in the popup window. Learners are required to complete a graphical view by moving gates to the correct positions according to the code given (see Figure 3(b)). Each line of the code that contains gates is highlighted. The students can tap on the highlighted code lines and then tap on the corresponding white box in the graphical representation to see how the correct gate moves to the correct place (see Figure 3(b)). Two hint boxes appear at the top right of the graphical representation and at the bottom of the code view to remind users touchable objects. When the user picks the correct pair of code line and white box, (i) the correct gate moves to the correct position in the graphical representation; (ii) a Boolean function and truth table are shown; (iii) inputs and outputs of each gate are demonstrated in green and red color, respectively. If a wrong pair of code and white box is chosen, a popup window appears.

¹We make our prototype and all source files available at <https://github.com/maziarser/LpLc>

4.1.3 Evaluation. Truth tables and Boolean functions are removed permanently in the "self-evaluation" section, and touchable inputs and gates take their places. Furthermore, learners need to remember the Boolean functions and truth tables to pick the correct inputs based on the final output (0/1) shown in the black/white boxes. The final output is selected randomly. At the beginning of the evaluation section, a popup window gives a textual explanation. The user can tap on the highlighted code lines and then tap on the corresponding objects to move them to the correct places in the graphical representation. The users can make connections between two objects in the graphical representation. The number of mistakes and the time are shown to the user (see Figure 3(c)). The system realizes the end of the performance based on the number of connections extracted from the Verilog code. For instance, if eight connections are extracted from the Verilog code, the end of performance is when the user makes eight connections between graphical objects. In this way, a hint box always appears at the top right of the graphical view to show learners that the system is waiting for the begin "choose the first object!" or end "choose the second object!" of a connection. Furthermore, two hint boxes appear at the bottom of the code view and movable objects to remind the user which objects are touchable. The final result is demonstrated in three formats: (i) when the user made a mistake in picking the right inputs based on the final output, (ii) When the user picks the right inputs but wrong connections, and (iii) When the user picks both the right inputs and connections. In the first two formats, system shows a popup window including information about input mistakes and about the number of connection errors (correct connections are marked); then it asks the user to try again. In the third format, a popup window gives information about total number of errors and time taken.

A "Reset" button is available to reset the whole graphical view and remove the connections. Touchable objects are shown in their first places. All changes are removed from the code view and the output is selected again. The number of errors will be reset to zero while the time is increasing continually.

5 CONCLUSION AND FUTURE WORK

In this paper we discussed four aspects of an m-learning application: (i) A method of representation of information for the learning topic "logic circuit design" is presented, (ii) users can interact with interactive multimedia elements, (iii) users are given real-time responses based on their inputs, and (iv) information is divided into three different sections ("tutorial", "practice", and "self-evaluation") and visualized dynamically based on 4C/ID instructional model.

A set of limitations restrained the designing and developing process. The main limitation is the screen and keypad size of mobile devices; thus, Verilog code describing a logic circuit is restricted to a manageable number of lines. The small keypad size of mobile devices induced a user interface preventing users' text inputs. Instructors also need to follow the learning system instructions to write the Verilog code in a way that the system is able to parse it.

In future work, the parser should be improved to derestrict the Verilog code guidelines and to allow instructors to write Verilog code in a more nondescript way. It also needs to be empirically investigated to what extent specific features of this learning system enhance learning outcomes, motivation, and acceptance on the part

of the users. In addition, while several studies indicate the benefits of multiple representations for learning outcomes, the issue of how to combine them in order to support learners to integrate them in one coherent mental model is of major importance. Thus, this is a major concern for future work to investigate the impacts of the mobile-based learning system on students' learning outcomes and on their attitudes (confidence, enjoyment, perceived difficulty, and interest in future learning opportunities) towards hardware description languages (in this case, Verilog). Furthermore, it is an interesting approach for future work to find out whether using a visual representation of concepts for logic circuits would have any effect on self-practice and motivation of students throughout hardware programming courses. Similarly, comparing the mobile-based learning system with web-based environments can be another pathway of future work to evaluate students' performance and attitude towards self-learning and self-practice in the context of hardware description languages.

This m-learning prototype was developed for Android-based mobile devices as a tool to improve independent practice in a blended e-learning environment. In this way, the four-component instructional design model 4C/ID was used to support learners. Multiple representations and interactivity are the two factors which are discussed in this paper to motivate users to practice independently.

REFERENCES

- [1] Shaaron Ainsworth. 2006. DeFT: A conceptual framework for considering learning with multiple representations. *Learning and instruction* 16, 3 (2006), 183–198.
- [2] Shaaron Ainsworth and Nicolas VanLabeke. 2004. Multiple forms of dynamic representation. *Learning and instruction* 14, 3 (2004), 241–255.
- [3] Stefan Belinov. 2021. Logic Circuit Simulator Pro (Version 21.0.1). Date February 06, 2021 from <https://play.google.com/store/apps/details?id=com.duracodefactory.logiccircuitsimulatorpro&gl=NL>.
- [4] Ben Cohen. 2012. *VHDL coding styles and methodologies*. Springer Science & Business Media.
- [5] Athanasios Drigas, Leyteris Koukianakis, and Yannis Papagerasimou. 2006. A web based e-learning and e-psychology modular environment. In *International Conference on Next Generation Web Services Practices*. IEEE, 168–174.
- [6] D Randy Garrison and Heather Kanuka. 2004. Blended learning: Uncovering its transformative potential in higher education. *The internet and higher education* 7, 2 (2004), 95–105.
- [7] Randall L Geiger, Phillip E Allen, and Noel R Strader. 1990. VLSI design techniques for analog and digital circuits. (1990).
- [8] hellotape. 2020. VHDL and Verilog (Version 5). Date February 07, 2020 from <https://play.google.com/store/apps/details?id=com.hellotape.hdl.vhdlandverilog>.
- [9] Intelitech. 2020. Digital Circuits (Version 2.4). Date August 19, 2020 from <https://play.google.com/store/apps/details?id=in.softecks.digitalcircuits&gl=NL>.
- [10] Santiram Kal. 2009. *Basic Electronics: Devices, Circuits and IT Fundamentals*. PHI Learning Pvt. Ltd.
- [11] Asif Irshad Khan, Mahaboob Sharief Shaikh, Abdullah Maresh Ali, Ch Vijaya Bebi, et al. 2012. Study of blended learning process in education context. *International Journal of Modern Education and Computer Science* 4, 9 (2012), 23.
- [12] Marguerite L Koole. 2009. A model for framing mobile learning. *Mobile learning: Transforming the delivery of education and training* 1, 2 (2009), 25–47.
- [13] Saija Lemmelä, Akos Vetek, Kaj Mäkelä, and Dari Trendafilov. 2008. Designing and evaluating multimodal interaction for mobile contexts. In *Proceedings of the 10th international conference on Multimodal interfaces*. 265–272.
- [14] Jian Li. 2010. Study on the development of mobile learning promoted by cloud computing. In *2010 2nd International Conference on Information Engineering and Computer Science*. IEEE, 1–4.
- [15] M Morris Mano and Michael D Ciletti. 2007. *Digital design*. Pearson Prentice Hall Upper Saddle River, NJ.
- [16] Paul Marshall. 2007. Do tangible interfaces enhance learning?. In *Proceedings of the 1st international conference on Tangible and embedded interaction*. 163–170.
- [17] Richard E Mayer and Roxana Moreno. 2002. Aids to computer-based multimedia learning. *Learning and instruction* 12, 1 (2002), 107–119.
- [18] Richard E Mayer and Valerie K Sims. 1994. For whom is a picture worth a thousand words? Extensions of a dual-coding theory of multimedia learning. *Journal of educational psychology* 86, 3 (1994), 389.

- [19] Mário Melo and Guilhermina Lobato Miranda. 2014. Applying the 4C-ID model to the design of a digital educational resource for teaching electric circuits: effects on student achievement. In *Proceedings of the 2014 Workshop on Interaction Design in Educational Environments*. 8–14.
- [20] Zainalabedin Navabi. 1999. *Verilog digital system design*. McGraw-Hill.
- [21] Pei-Luen Patrick Rau, Qin Gao, and Li-Mei Wu. 2008. Using mobile communication technology in high school education: Motivation, pressure, and learning performance. *Computers & Education* 50, 1 (2008), 1–22.
- [22] M. Seraj, C.S. Große, and R. Drechsler. 2017. Self-practicing of logic circuits through mobile devices: Lecturers' and students' perceptions. In *EDULEARN17 Proceedings* (Barcelona, Spain) (*9th International Conference on Education and New Learning Technologies*). IATED, 187–196. <https://doi.org/10.21125/edulearn.2017.1045>
- [23] Maziar Seraj and Chui Yin Wong. 2012. A study of User Interface Design principles and requirements for developing a Mobile learning prototype. In *2012 International Conference on Computer & Information Science (ICCIS)*, Vol. 2. IEEE, 1014–1019.
- [24] Maziar Seraj and Chui Yin Wong. 2014. Impacts of Different Mobile User Interfaces on Student Satisfaction for Learning Dijkstra's Shortest Path Algorithm. *International Journal of Interactive Mobile Technologies* 8, 4 (2014).
- [25] Maziar Seraj and Chui Yin Wong. 2014. Lecturers and Students' Perception on Learning Dijkstra's Shortest Path Algorithm Through Mobile Devices. *International Journal of Interactive Mobile Technologies* 8, 3 (2014).
- [26] Martin Spindler, Christian Tominski, Heidrun Schumann, and Raimund Dachsel. 2010. Tangible views for information visualization. In *ACM International Conference on Interactive Tabletops and Surfaces*. 157–166.
- [27] Astrid Pratidina Susilo, Jeroen van Merriënboer, Jan van Dalen, Mora Claramita, and Albert Scherpbier. 2013. From lecture to learning tasks: use of the 4C/ID model in a communication skills course in a continuing professional education context. *The Journal of Continuing Education in Nursing* 44, 6 (2013), 278–284.
- [28] Donald Thomas and Philip Moorby. 2008. *The Verilog® hardware description language*. Springer Science & Business Media.
- [29] Barbara Tversky, Julie Bauer Morrison, and Mireille Betrancourt. 2002. Animation: can it facilitate? *International journal of human-computer studies* 57, 4 (2002), 247–262.
- [30] Jeroen JG Van Merriënboer, Richard E Clark, and Marcel BM De Croock. 2002. Blueprints for complex learning: The 4C/ID-model. *Educational technology research and development* 50, 2 (2002), 39–61.
- [31] Wrapptec. 2021. Smart Logic Simulator (Version 7.0.3). Date February 03, 2021 from <https://play.google.com/store/apps/details?id=com.tomaszczart.smartlogicsimulator&gl=NL>.